

International University

School of Computer Science and Engineering

Web Application Development

Laboratory

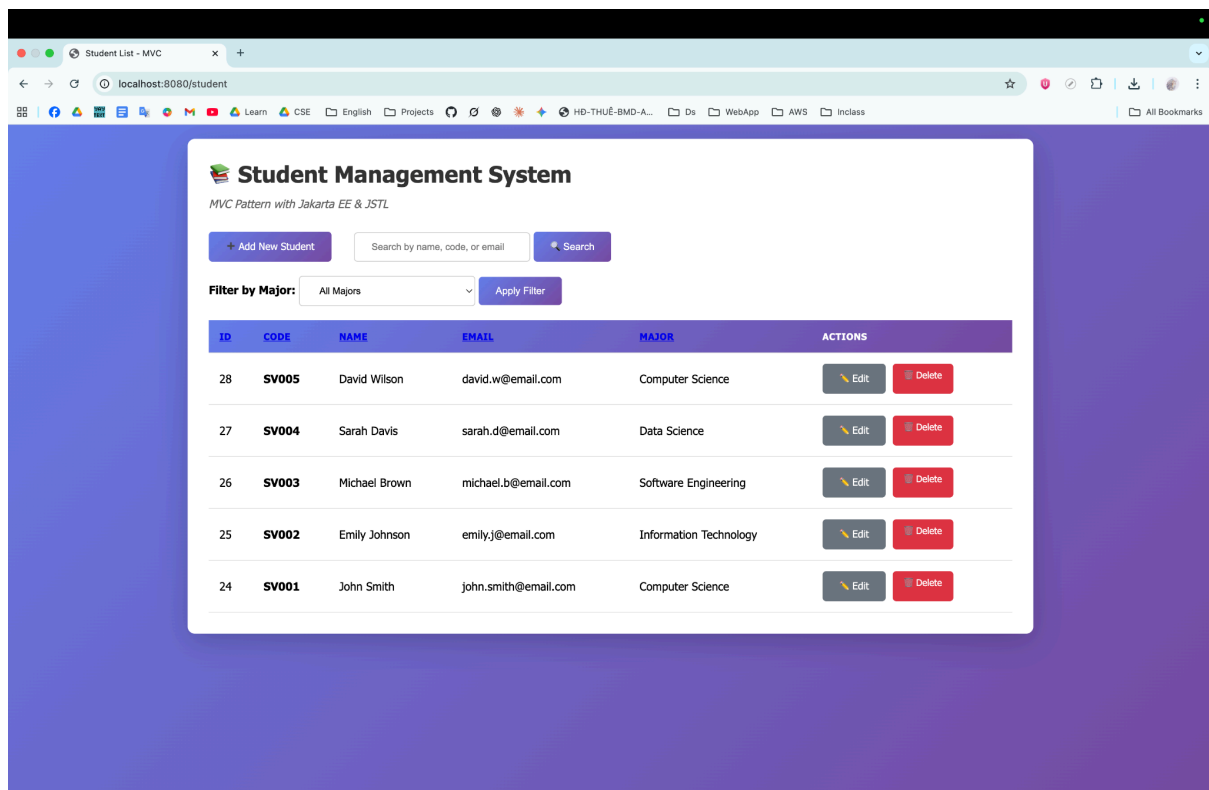
IT093IU

Lab #5

Submitted by

Nguyễn Hồng Ngọc Hân - ITCSIU22229

Home page with student list



Explain:

When the user goes to `/student`, the browser sends a GET request to the controller. The `doGet()` method receives this request and sees that the action is "list," so it calls `listStudents()`. This method asks the DAO to get the student records from the database. After the data is returned, the controller stores the list inside the request using `setAttribute`, so the JSP page can read it. Instead of redirecting the browser, the controller forwards the request to `student-list.jsp`. The URL stays the same, but the JSP receives the student list and shows it on the page. In summary, the controller gets the data, puts it into the request, and sends it to the JSP for display.

Add student form

The screenshot shows a web browser window with the address bar displaying `localhost:8080/student?action=new`. The browser's taskbar at the bottom lists various applications including 'Learn', 'CSE', 'English', 'Projects', 'HD-THUE-BMD-A...', 'Ds', 'WebApp', 'AWS', and 'Inclass'. The main content area features a purple gradient background with a white modal form titled 'Add New Student'. The form includes the following fields and controls:

- Student Code ***: A text input field containing the placeholder text 'e.g., SV001, IT123'. Below the field is a hint: 'Format: 2 letters + 3+ digits'.
- Full Name ***: A text input field with the placeholder text 'Enter full name'.
- Email ***: A text input field containing the placeholder text 'student@example.com'.
- Major ***: A dropdown menu with the text '-- Select Major --' and a downward arrow.
- Buttons**: Two buttons at the bottom of the form: a blue button with a plus icon and the text 'Add Student', and a grey button with a red 'X' icon and the text 'Cancel'.

Explain:

When the user clicks “Add New Student,” the controller receives the action new and forwards the request to student-form.jsp. The JSP loads the form in create mode, which means no existing data is pre-filled and the page shows blank input fields. The form includes fields for student code, full name, email, and major, along with basic validation rules such as required inputs and format hints. When the user submits the form, the data is sent to /student using a POST request with the action insert. The controller then reads all the fields, validates them, and passes the values to the DAO to insert a new record into the database. If the insertion is successful, the user is redirected back to the student list page with a success message.

Edit student form (pre-filled)

Edit Student

Student Code *
SV005
Format: 2 letters + 3+ digits

Full Name *
David Wilson

Email *
david.w@email.com

Major *
Information Technology

[Update Student](#) [Cancel](#)

Student Management System
MVC Pattern with Jakarta EE & JSTL

✓ Student updated successfully

[+ Add New Student](#) [Search](#)

Filter by Major: All Majors [Apply Filter](#)

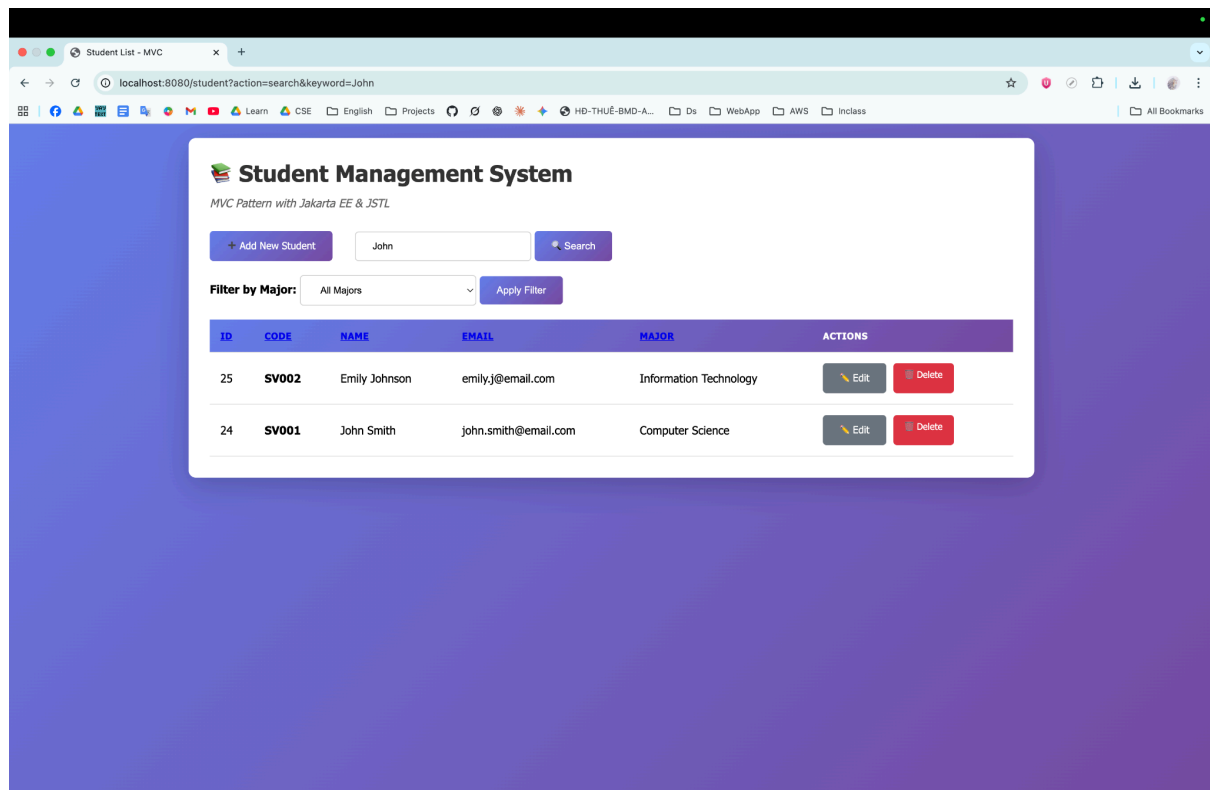
ID	CODE	NAME	EMAIL	MAJOR	ACTIONS
28	SV005	David Wilson	david.w@email.com	Information Technology	Edit Delete
27	SV004	Sarah Davis	sarah.d@email.com	Data Science	Edit Delete
26	SV003	Michael Brown	michael.b@email.com	Software Engineering	Edit Delete
25	SV002	Emily Johnson	emily.j@email.com	Information Technology	Edit Delete
24	SV001	John Smith	john.smith@email.com	Computer Science	Edit Delete

Explain:

When the user clicks the Edit button on a student record, the controller receives the action edit along with the student's ID. It then calls `showEditForm()` and uses the DAO to fetch that student's information from the database. Once the data is retrieved, the controller stores the Student object in the request using `setAttribute` and forwards the request to `student-form.jsp`. Because the form is opened in edit mode, all fields—such as student code, name, email, and major—are already filled with the existing data. This allows the user to update only the information they want to change. When the form is submitted, a POST request with

the action update is sent to the controller, which performs validation and then calls the DAO to update the record in the database. After a successful update, the controller redirects the user back to the student list page and displays a success message at the top.

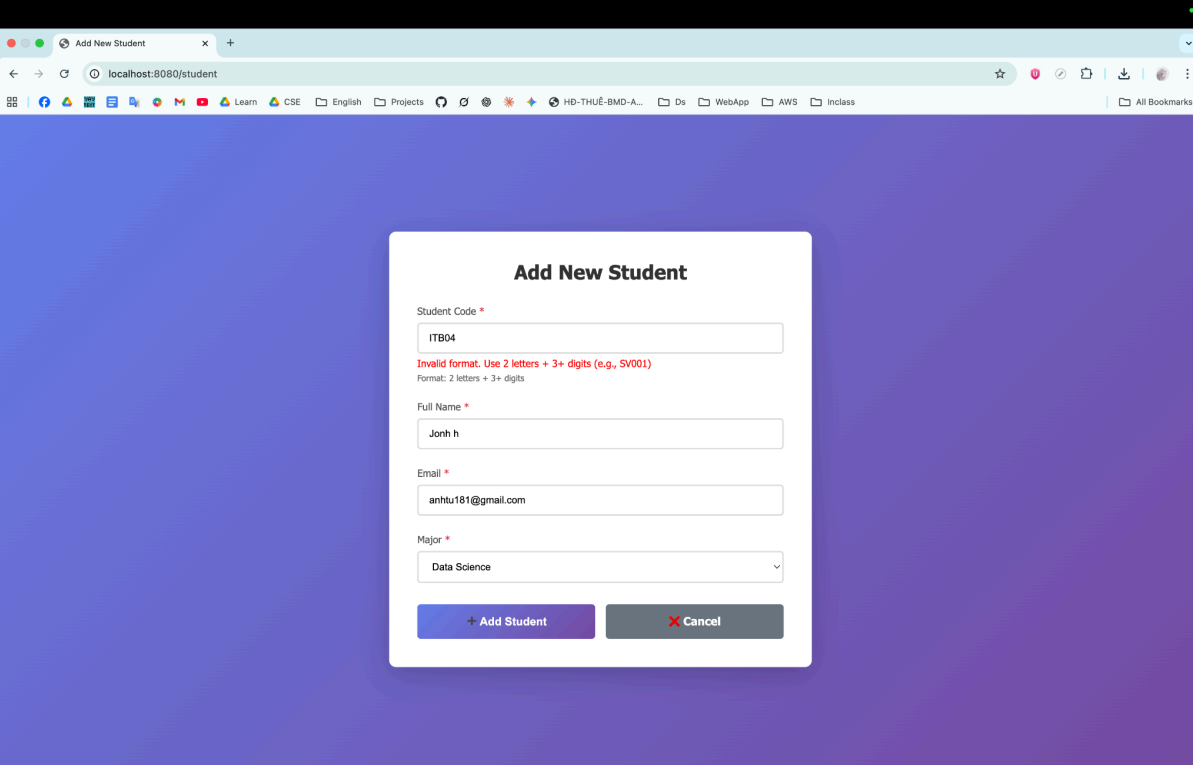
Search results



Explain:

When the user enters a keyword into the search box and submits the form, the browser sends a GET request with the action search and the typed keyword. The controller receives this request and calls `searchStudents()`, which checks whether the keyword is empty. If the user provided a keyword, the DAO runs a query that searches student code, full name, and email using a LIKE pattern. The matching results are returned and stored in the request using `setAttribute`, allowing the JSP to access them. The controller then forwards the request to `student-list.jsp`, where the page displays only the students that match the search term. The keyword is also kept in the input field so the user can easily adjust their search. In short, the search feature filters the student list based on the user's input and shows only relevant results.

Validation errors

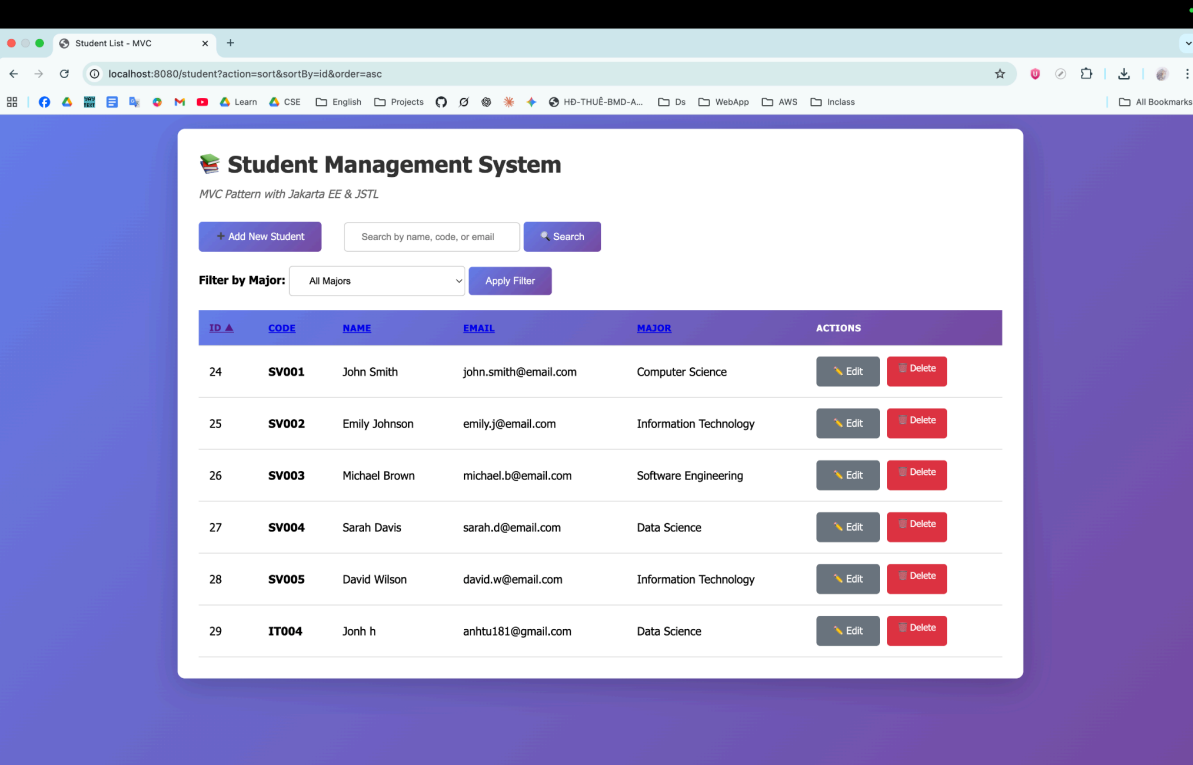


The screenshot shows a web browser window with the address bar displaying 'localhost:8080/student'. The page has a purple gradient background. In the center is a white modal form titled 'Add New Student'. The form contains four input fields: 'Student Code', 'Full Name', 'Email', and 'Major'. The 'Student Code' field contains 'ITB04' and has a red error message below it: 'Invalid format. Use 2 letters + 3+ digits (e.g., SV001)' and 'Format: 2 letters + 3+ digits'. The 'Full Name' field contains 'Jonh h'. The 'Email' field contains 'anhthu181@gmail.com'. The 'Major' field is a dropdown menu with 'Data Science' selected. At the bottom of the form are two buttons: a purple 'Add Student' button and a grey 'Cancel' button.

Explain:

When the user submits the Add Student form, the controller receives the POST request and runs the `validateStudent()` method to check all input fields. If any value does not meet the required rules—such as an invalid student code format, an empty name, or an incorrect email pattern—the validation fails. Instead of inserting the data, the controller places the error messages into the request using `setAttribute` and forwards the user back to the form. Because the form is reloaded with the existing input values, the user can immediately see what needs to be corrected without typing everything again. The error message is displayed next to the specific field, clearly indicating what is wrong. In short, the system prevents invalid data from being saved and guides the user to fix mistakes before submitting again.

Sorted list



Student Management System
MVC Pattern with Jakarta EE & JSTL

+ Add New Student Search by name, code, or email Search

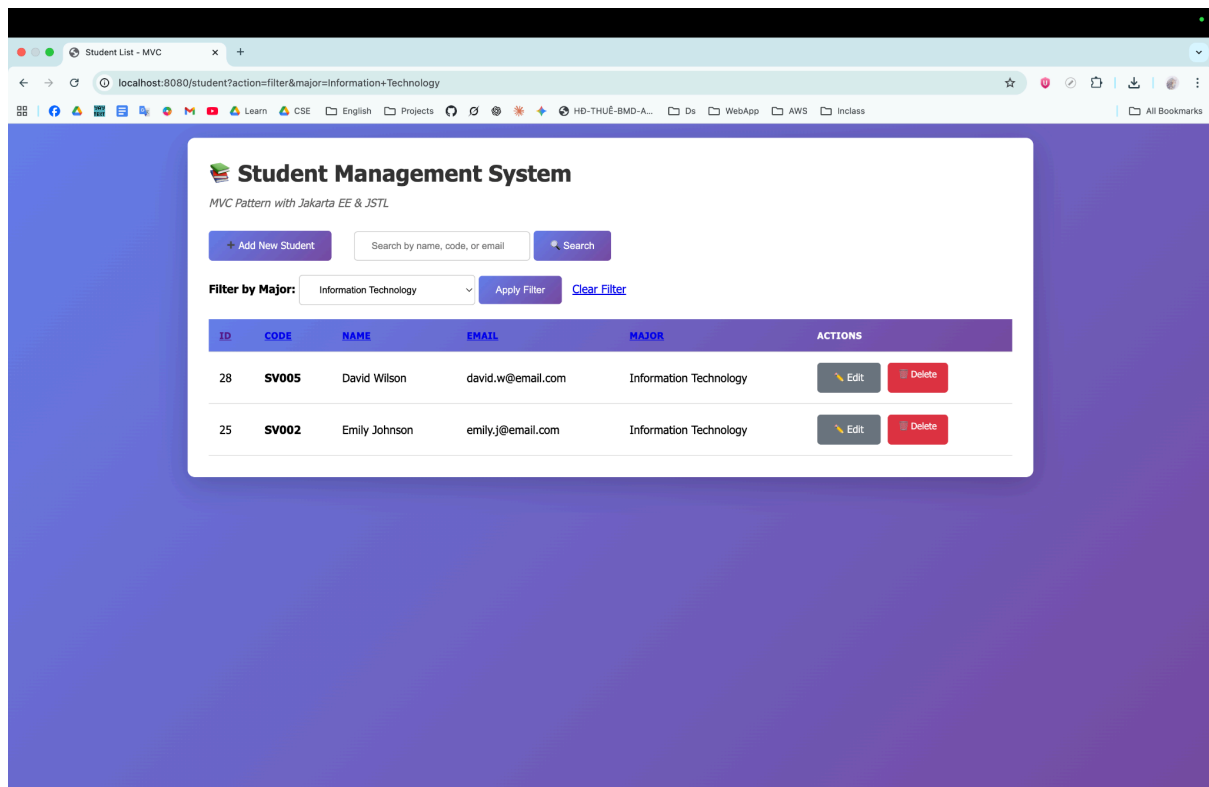
Filter by Major: All Majors Apply Filter

ID	CODE	NAME	EMAIL	MAJOR	ACTIONS
24	SV001	John Smith	john.smith@email.com	Computer Science	Edit Delete
25	SV002	Emily Johnson	emily.j@email.com	Information Technology	Edit Delete
26	SV003	Michael Brown	michael.b@email.com	Software Engineering	Edit Delete
27	SV004	Sarah Davis	sarah.d@email.com	Data Science	Edit Delete
28	SV005	David Wilson	david.w@email.com	Information Technology	Edit Delete
29	IT004	Jonh h	anhtu181@gmail.com	Data Science	Edit Delete

Explain:

When the user clicks a column header, such as Code or Name, the page sends a GET request with the action sort along with two parameters: sortBy (the selected column) and order (ascending or descending). The controller receives these values and calls `sortStudents()`, which validates the parameters and asks the DAO to return the student list sorted accordingly. Once the sorted data is retrieved, the controller stores the results in the request using `setAttribute` and forwards everything to `student-list.jsp`. The JSP then displays the students in the chosen order and shows an arrow icon to indicate the current sorting direction. In short, the sorting feature lets users reorganize the table by any column, making it easier to find and compare student information.

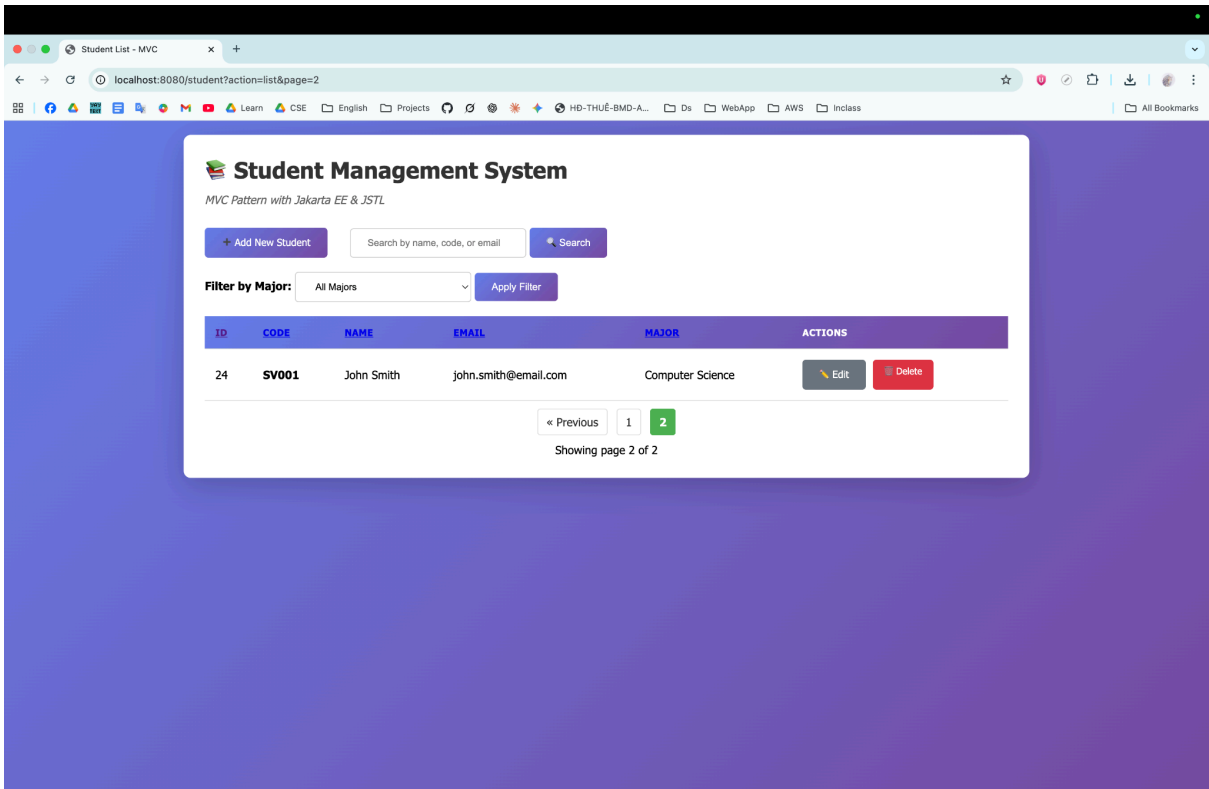
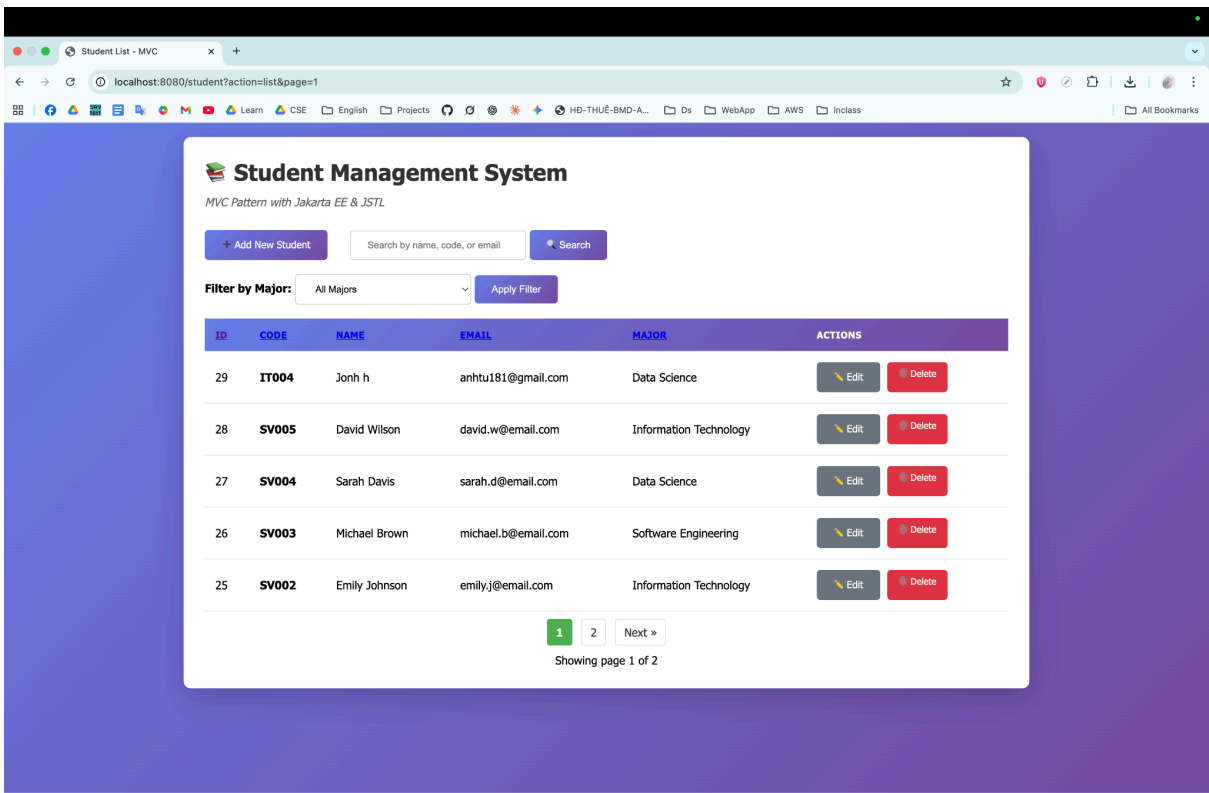
Filtered list



Explain:

When the user chooses a major from the dropdown and clicks Apply Filter, the browser sends a GET request with the action filter and the selected major. The controller receives these values and calls `filterStudents()`, which checks whether a major was provided. If a specific major is selected, the DAO returns only the students whose major matches that value; otherwise, it returns the full list. The controller then stores the filtered results and the selected major into the request using `setAttribute` and forwards everything to `student-list.jsp`. The JSP displays only the students that belong to the chosen major and highlights the filter with a "Clear Filter" link to reset the view. In short, the filtering feature lets users quickly narrow the list to students from a specific major.

PAGINATION



Explain:

When the user opens the student list, the controller loads only a limited number of records per page—in this case, five students. The controller reads the current page number from the URL and calculates the correct offset for the database query. It then asks the DAO for the specific subset of students that belong to that page, along with the total number of records. After retrieving the data, the controller stores the student list, current page, and total pages inside the request and forwards everything to `student-list.jsp`. The JSP uses this information to display page numbers, along with “Previous” and “Next” buttons. Clicking any of these sends a new GET request with the updated page number, allowing the user to navigate through the list.